

คู่มือ Claude Code — ฉบับสอนครบชุด

คำสั่ง built-in (หลายมิติ: ทำอะไร → ใช้เมื่อไร → บริบทจริง) · โหมด permission (`shift + tab`) · วิธี pin โมเดล (เช่น Opus 4.6) · วิธีตั้ง status line หลายแถว

อ่านตารางยังงี้: ช่อง **ใช้เมื่อไร-เป้าหมาย** คือ "อยากรู้เรื่องคำสั่งนี้" · ช่อง **ตัวอย่าง** คือสถานการณ์จริงที่เจอบ่อย เอาไปเล่าให้ผู้เรียนเห็นภาพได้เลย

01 เริ่มต้น & ขอความช่วยเหลือ				คำสั่งแรกที่ควรสอน
คำสั่ง	ทำอะไร	ใช้เมื่อไร-เป้าหมาย	ตัวอย่างสถานการณ์	
<code>/help</code>	ดูคำสั่งทั้งหมด	นึกคำสั่งไม่ออก อยากรู้เมนูรวม	เพิ่งเปิดใช้ครั้งแรก พิมพ์ <code>/help</code> ดูว่าทำอะไรได้บ้าง	
<code>/status</code>	ดูเวอร์ชัน โมเดล บัญชีที่ล็อกอิน	อยากรู้ว่ากำลังใช้โมเดลอะไร บัญชีไหน	งงว่าทำไมช้า/แพง เช็คที่ใช้ Opus หรือ Sonnet อยู่	
<code>/doctor</code>	ตรวจสอบสุขภาพการติดตั้ง	มีอะไรพัง อยากรู้สาเหตุ	MCP ไม่ทำงาน/ต่อเน็ตไม่ได้ รับ <code>/doctor</code> หาจุดเสีย	
<code>/release-notes</code>	ดูของใหม่ที่เพิ่งอัปเดต	อัปเดตแล้วอยากรู้มีอะไรเพิ่ม	เห็นปุ่มใหม่ในเมนู เช็คว่ามีฟีเจอร์อะไรมาใหม่	

02 จัดการสนทนา & context			หัวใจของการใช้ให้คุ้ม
คำสั่ง	ทำอะไร	ใช้เมื่อไร-เป้าหมาย	ตัวอย่างสถานการณ์
/clear = /new /reset	ล้างแชท เริ่มใหม่หมด	เปลี่ยนเรื่องคุย ไม่อยากให้เรื่องเก่ามากวน/เปลี่ยนโทน	คุยงาน A เสร็จ จะเริ่มงาน B /clear ก่อน กันมันจำเรื่องเก่าผิดๆ
/compact	บีบอัดบทสนทนาให้สั้น (เก็บใจความ)	คุยยาวมาก context ใกล้เคียง แต่ยังไม่อยากทำงานต่อ	แก้ไ่คิดมาเป็นชั่วโมง เริ่มช้า /compact ให้มันสรุปแล้วไปต่อ
/context	ดูว่าใช้ context ไปเท่าไรแล้ว	อยากรู้ว่าเหลือที่ให้คุยอีกแค่ไหน	รู้สึกมันเริ่มลืม เช็คว่าเต็มหรือยังจะได้ /compact กัน
/resume = /continue	กลับไปทำงานที่คุยค้างไว้	ปิดไปแล้ว อยากสานต่อของเดิม	เมื่อวานแก้บั๊กค้าง /resume เลือกแชทเดิมมาทำต่อ
/rewind = /undo /checkpoint	ย้อนกลับไปจุดก่อนหน้า	มันทำพลาด อยากถอยกลับก่อนพัง	Claude แก้ผิดไฟล์ /rewind ย้อนกลับจุดที่ยังดีอยู่
/export	บันทึกบทสนทนาเป็นไฟล์	อยากเก็บบันทึก/ส่งต่อให้คนอื่นดู	คุยได้คำตอบดีๆ /export เก็บไว้เป็นเอกสาร

03 โมเดล & การตั้งค่า			คุมพลัง คุมความปลอดภัย
คำสั่ง	ทำอะไร	ใช้เมื่อไร-เป้าหมาย	ตัวอย่างสถานการณ์
/model	สลับโมเดล AI	อยากได้ฉลาดขึ้น (Opus) หรือเร็ว/ประหยัดขึ้น (Sonnet/Haiku)	งานคิดหนัก สลับเป็น Opus · งานง่ายๆ ใช้ Haiku ประหยัด
/config	เปิดหน้าต่างค่า (รีโมเดล ฯลฯ)	อยากปรับหน้าต่าง/ค่าเริ่มต้น	อยากเปลี่ยนธีมมืด-สว่าง เข้า /config ปรับ
/permissions = /allowed-tools	ตั้งกฎว่าให้รันอะไรได้บ้าง	อยากลดการถาม yes/no ช้าๆ กับคำสั่งที่ไร้ใจ	มันถาม npm ทุกครั้ง /permissions อนุญาต npm ถาวร
/vim	เปิดโหมดพิมพ์แบบ vim	ถนัด vim อยากใช้คีย์ลัดแบบเดิม	คนคุ้น vim เปิดไว้พิมพ์เร็วขึ้น

04 โปรเจกต์ & ความจำ			ทำให้ Claude เข้าใจงานเรา
คำสั่ง	ทำอะไร	ใช้เมื่อไร—เป้าหมาย	ตัวอย่างสถานการณ์
<code>/init</code>	สร้างไฟล์ CLAUDE.md อธิบายโปรเจกต์	เริ่มใช้ Claude กับ repo ใหม่ อยากให้มันรู้จักโครงสร้าง	โคลนโปรเจกต์มา <code>/init</code> ให้มันสแกน แล้วเขียนคู่มือให้ตัวเอง
<code>/memory</code>	แก้ไฟล์ความจำถาวร	อยากให้มันจำสไตล์/กฎของเรา ชำนาญ	อยากให้ตอบไทยเสมอ <code>/memory</code> เพิ่มกฎลงไป
<code>/add-dir</code>	เพิ่มโฟลเดอร์ให้เข้าถึงไฟล์	ต้องใช้ไฟล์ที่อยู่นอกโฟลเดอร์ปัจจุบัน	โค้ดอยู่คนละที่กับ config <code>/add-dir</code> เพิ่มอีกโฟลเดอร์

05 งานโค้ด & รีวิว			สายพัฒนาใช้บ่อย
คำสั่ง	ทำอะไร	ใช้เมื่อไร—เป้าหมาย	ตัวอย่างสถานการณ์
<code>/plan</code>	เข้าโหมดวางแผนก่อนลงมือ	งานใหญ่/ซับซ้อน อยากเห็นแผนก่อน ให้มันแก้จริง	จะรี้อไฟเจอร์ใหญ่ <code>/plan</code> ให้มันเสนอแผนก่อนอนุมัติ
<code>/code-review</code>	รีวิวโค้ดที่เพิ่งแก้ หาบั๊ก	แก้เสร็จ อยากให้ช่วยตรวจสอบก่อน commit	เขียนฟังก์ชันใหม่ <code>/code-review</code> เช็ค edge case ที่ลืม
<code>/security-review</code>	ตรวจสอบช่องโหว่ความปลอดภัย	ก่อนขึ้น production อยากกันเรื่องความปลอดภัย	มีฟอร์มรับข้อมูลผู้ใช้ <code>/security-review</code> หา injection/รี้ว
<code>/review</code>	รีวิว Pull Request บน GitHub	มี PR รอ อยากให้ช่วยดูก่อน merge	เพื่อนส่ง PR มา <code>/review</code> ดูว่าปลอดภัยที่จะ merge ไหม

06 เครื่องมือ & การเชื่อมต่อ			ต่อยอด ขยายความสามารถ
คำสั่ง	ทำอะไร	ใช้เมื่อไร-เป้าหมาย	ตัวอย่างสถานการณ์
<code>/mcp</code>	จัดการ MCP server (เครื่องมือเสริม)	อยากต่อ Claude เข้ากับ Chrome/Google Drive/DB ฯลฯ	จะให้คุ่มเบราว์เซอร์ <code>/mcp</code> เช็ค/รีคอนเนกต์ Chrome MCP
<code>/agents</code>	จัดการ subagent (ผู้ช่วยย่อย)	อยากให้มีผู้ช่วยเฉพาะทางแยกงาน	งานหลายด้านพร้อมกัน <code>/agents</code> ตั้งผู้ช่วยเฉพาะเรื่อง
<code>/hooks</code>	ดู/ตั้ง automation อัตโนมัติ	อยากให้ทำอะไรเองทุกครั้งที่เกิดเหตุการณ์ X	อยากให้เปิดไฟล์ผลลัพธ์อัตโนมัติ <code>/hooks</code> ตั้ง Stop hook

07 บัญชี & ค่าใช้จ่าย			
คำสั่ง	ทำอะไร	ใช้เมื่อไร-เป้าหมาย	ตัวอย่างสถานการณ์
<code>/usage</code> = <code>/cost</code> <code>/stats</code>	ดูยอดใช้งาน/ค่าใช้จ่าย	อยากรู้ว่าใช้ไปเท่าไร เหลือโควตาแค่ไหน	ใกล้สิ้นเดือน <code>/usage</code> เช็คค่าใช้จ่ายที่ % ของแพ็คเกจ
<code>/login</code> / <code>/logout</code>	เข้า/ออกจากบัญชี	สลับบัญชี หรือเครื่องใหม่	เปลี่ยนไปใช้บัญชีบริษัท <code>/logout</code> แล้ว <code>/login</code> ใหม่
<code>/feedback</code> = <code>/bug</code>	รายงานปัญหา/ส่งความเห็น	เจอบั๊ก อยากแจ้งทีม Anthropic	คำสั่งทำงานเพี้ยน <code>/bug</code> ส่ง log ให้ทีมดู

Plan mode

อ่าน/วิเคราะห์อย่างเดียว — ไม่แก้ไฟล์ ไม่รันอะไรที่เปลี่ยนแปลง จะเสนอแผนมาให้ดูก่อน รออนุมัติค่อยลงมือ

ใช้เมื่อ: งานใหญ่/สำคัญ/production · อยากให้คิดก่อนทำ กันพลาด · เหมาะกับมือใหม่

เสียงต่ำสุด

Default

โหมดปกติ — ถามทุกครั้งก่อนแก้ไฟล์หรือรันคำสั่ง คุณกด yes/no เอง

ใช้เมื่อ: อยากคุมทุกก้าว · งานทั่วไปที่ยังไม่ไว้วางใจ

เสียงต่ำ

accept edits

อนุมัติการแก้ไฟล์ให้อัตโนมัติ (ไม่ต้องกด yes ทุกครั้ง) — แต่ยังถามก่อนรันคำสั่ง shell ที่เสี่ยง

ใช้เมื่อ: งานแก้ไขโค้ดเยอะๆ ที่ไว้วางใจได้ อยากให้ไหลลื่น แต่ยังคงคุมคำสั่งอันตราย

เสียงกลาง

auto mode

อนุมัติเกือบทุกอย่างเอง รวมถึงรันคำสั่งด้วย — ทำงานต่อเนื่อง ไม่ค่อยหยุดถาม

ใช้เมื่อ: งานที่ไหลยาว อยากให้เดินเองจนจบ · เชื่อมือแล้ว

เสียงสูง

bypass permissions

ไม่ถามอะไรเลย อนุมัติทุกอย่าง รวมถึงคำสั่งลบไฟล์/อันตราย — เร็วสุด แต่มันรันอะไรก็ได้โดยไม่เตือน

ใช้เมื่อ: มั่นใจจริงๆ หรืออยู่ใน sandbox/เครื่องแยกเท่านั้น · ไม่แนะนำกับมือใหม่ หรือเครื่องที่มีของสำคัญ

เสียงสูงสุด

โหมด	แก้ไฟล์เอง?	รันคำสั่งเอง?	ความเสี่ยง
Plan	ไม่	ไม่	ต่ำสุด
Default	ถามก่อน	ถามก่อน	ต่ำ
accept edits	ใช่	ถามก่อน	กลาง
auto	ใช่	ใช่ (เกือบหมด)	สูง
bypass	ใช่	ใช่ (ทั้งหมด)	สูงสุด

จำง่ายๆ: สีเขียว = ระวังสุด · สีแดง = ปล่อยสุด — กด **shift + tab** วนไปเรื่อยๆ จนเจอโหมดที่อยากใช้

+ คำสั่ง built-in อื่นๆ ที่ควรรู้

ยังมีอีก รวมทั้งหมด ~45 คำสั่ง

/effort — ตั้งระดับความคิดหนัก (low→max)

/rewind — ย้อน checkpoint

/copy — คัดลอกคำตอบล่าสุด

/cd — ย้ายโฟลเดอร์ทำงาน

/diff — ดูความต่างของโค้ด

/skills — ดู skill ที่มี

/ide — เชื่อมกับ VS Code / JetBrains

/terminal-setup — ตั้งคีย์ลัด terminal

/plugin — จัดการปลั๊กอิน

/loop — รันคำสั่งซ้ำเป็นรอบ

/background — แยกงานไปรันเบื้องหลัง

/powerup — บกเรียนพีเจอร์แบบโต้ตอบ

หลักการ: `/model` เลือกได้ก็จริง แต่ค่าใน `settings.json` ชนะทุกครั้งตอน restart — ถ้า pin ผิดก็จะรู้สึกว่า "มันดีด ไม่ยอมเปลี่ยน"

ข้อจำกัด: เมนู `/model` เป็นลิสต์ตายตัวฝังในแอป (Default/Opus/Fable/Sonnet/Haiku) — เพิ่มโมเดลรุ่นเก่าเป็นบัสสก็อตให้เลือกไม่ได้ แต่ pin ทำให้มันเป็นตัวที่ **รันจริงทุกครั้ง** สลับชั่วคราวด้วย `/model claude-opus-4-6[1m]`

แบบ A — Prompt (วางในแชท Claude Code เครื่องไหนก็ได้)

ช่วยตั้งค่าให้ Claude Code ใช้โมเดล Opus 4.6 (1M context) เป็นค่าเริ่มต้นถาวร ทำตามนี้เป๊ะ ห้ามแก้ไฟล์อื่น ห้ามลบอะไร

- สำรอง `~/.claude/settings.json` เป็น `settings.json.bak` ก่อน (ถ้ายังไม่มีไฟล์ ให้สร้างใหม่เป็น `{}`)
- ตั้ง key `"model" = "claude-opus-4-6[1m]"` — มีแล้วแก้ค่า ยังไม่มีให้เพิ่ม คง key อื่นไว้
- เช็คว่ามี `~/.claude/settings.local.json` ที่มี key `"model"` มาทับ ถ้ามีให้ลบเฉพาะ key นั้น
- ทดสอบว่ายังติด: `claude --model 'claude-opus-4-6[1m]' -p "reply OK"`
- สรุปว่าตั้งที่ไฟล์ไหน ค่าเก่าคืออะไร และบอกให้ restart

ถ้าเจอ error ว่าไม่มีโมเดลนี้ ให้เปลี่ยนเป็น `"claude-opus-4-6"` (ไม่มี `[1m]`) แล้วลองใหม่

แบบ B — คำสั่งเดียว (Terminal ตรงๆ ไม่ต้องพึ่ง AI)

```
python3 - <<'PY'
import json, os, shutil
f = os.path.expanduser("~/claude/settings.json")
os.makedirs(os.path.dirname(f), exist_ok=True)
data = {}
if os.path.isfile(f):
    shutil.copy2(f, f + ".bak")          # สำรองก่อน
    try: data = json.load(open(f))
    except: data = {}
old = data.get("model", "(ยังไม่มี)")
data["model"] = "claude-opus-4-6[1m]"    # ← เปลี่ยนโมเดลตรงนี้
json.dump(data, open(f, "w"), indent=2, ensure_ascii=False)
print(f"pinned {f} : {old} → {data['model']} (backup {f}.bak)")
PY
claude --model 'claude-opus-4-6[1m]' -p "reply OK" # ทดสอบว่ายังติด
```

หลังตั้ง: ต้อง restart · เช็คด้วย `/status` · ถอยกลับ `cp ~/.claude/settings.json.bak ~/.claude/settings.json`
ไม่มีแพ็ก 1M? ตัด `[1m]` เหลือ `claude-opus-4-6` · ต่อหลาย terminal? ต้อง pin ให้ครบทุก config dir ที่ใช้จริง

10 วิธีตั้ง Status line หลากหลาย

โมเดล + usage + git

หลักการ: ตั้ง `statusLine` แบบ `type:"command"` → Claude Code ป้อน JSON ข้อมูล session เข้า stdin ของสคริปต์ → เอาข้อความที่ print ไปแสดง แต่ละบรรทัด = 1 แถว

```
Opus 4.6 (1M context) ██████████ 15% | 150k/1.0m
Current ██████████ 15% | Jul 10, 6:10 PM
Weekly ██████████ 14% | Jul 12, 8:00 PM
📁 aexgee ↗ main*
```

วิธี 1 — Installer ไฟล์เดียว (แนะนำ)

ก็อป `claude-statusline-installer.sh` ไปเครื่องอื่นแล้วรัน — ลง jq + เขียนสคริปต์ + ต่อ config + สำรอง .bak คนในทีม

```
bash claude-statusline-installer.sh
```

วิธี 2 — Prompt (ให้ AI ติดตั้งให้)

ช่วยตั้ง status line หลากหลายให้ Claude Code ห้ามแตะไฟล์อื่น:

- เช็ค/ลง jq ถ้ายังไม่มี
- สร้าง `~/.claude/statusline.sh` อ่าน JSON จาก stdin แล้ว print 4 แถว:
 - แถว1: ชื่อโมเดล + ขนาด context + progress bar + % + tokens (`.model.display_name`, `.context_window`)
 - แถว2: "Current" = usage 5 ชม. + เวลาที่เหลือ (`.rate_limits.five_hour`)
 - แถว3: "Weekly" = usage 7 วัน (`.rate_limits.seven_day`)
 - แถว4: โฟลเดอร์ + git branch + * ถ้ามีไฟล์ค้าง (`.workspace.current_dir`)ใช้ jq ดึงค่า, bar ใช้ `█/▒` กว้าง 20, รองรับ date ทั้ง macOS (-r) และ Linux (-d)
- `chmod +x`
- เพิ่ม key `statusLine` ใน `~/.claude/settings.json` = `{"type":"command","command":"bash ~/.claude/statusline.sh","padding":0}`
- บอกให้ restart

วิธี 3 — Manual (3 ขั้น)

1. ลง jq — `brew install jq` (Mac) / `sudo apt install jq` (Linux) 2. วางสคริปต์ที่ `~/.claude/statusline.sh` แล้ว `chmod +x` 3. ต่อ config:

```
{ "statusLine": { "type": "command", "command": "bash ~/.claude/statusline.sh", "padding": 0 }
```

เตือนผู้เรียน: ต้อง restart ถึงเห็นผล · ต้องมี jq (เช็ค `jq --version`) ไม่ขึ้นแถวว่าง · แถว Current/Weekly โผล่เฉพาะแพ็คเกจที่มี rate limit (Max/Team) · แถว "📁 N files" ต้องมี hook แยก ไม่มีก็ข้ามเฉยๆ

คู่มือคำสั่ง Claude Code (ฉบับสอน) · เฉพาะคำสั่ง built-in ที่มากับทุกเครื่อง

คำสั่งอย่าง `/Chrome-MCP-OWLy` · `/CRUD-basic` · `/codex:*` เป็นของที่ใช้เซตเพิ่มเอง (custom skill / plugin) ไม่ได้ติดมากับพื้นฐาน

อ้างอิงทางการ: code.claude.com/docs/en/commands